

Lecture 5: Error handling

Review tests, learn error handling, unsafe Rust

Willem Vanhulle

DevLab Rust 2025

Tuesday December 2, 2025

github.com/wvhulle/rust-course-ghent

1. Review tests	1
1.1. Events coming up	2
1.2. New tools	3
2. Error handling	4
3. Unsafe Rust	20
4. Homework	32

Allocated time: 15 min

1.1. Events coming up

Scripting with Nu Shell:

- Target audience: DevOps, sysadmins, backend developers
- Date: **tomorrow**, Wednesday December 3, 2025
- Time: 19:00 - 20:00
- Location: Kammerstraat 10, 9000 Gent
- Registration: <https://www.meetup.com/sysghent/events/311799711>

```
1 ps | where name == Notepad2.exe | get pid.0 | kill $in
```

(playground link)

1.1. Events coming up

Scripting with Nu Shell:

- Target audience: DevOps, sysadmins, backend developers
- Date: **tomorrow**, Wednesday December 3, 2025
- Time: 19:00 - 20:00
- Location: Kammerstraat 10, 9000 Gent
- Registration: <https://www.meetup.com/sysghent/events/311799711>

```
1 ps | where name == Notepad2.exe | get pid.0 | kill $in
```

(playground link)

WebAssembly for secure systems and services:

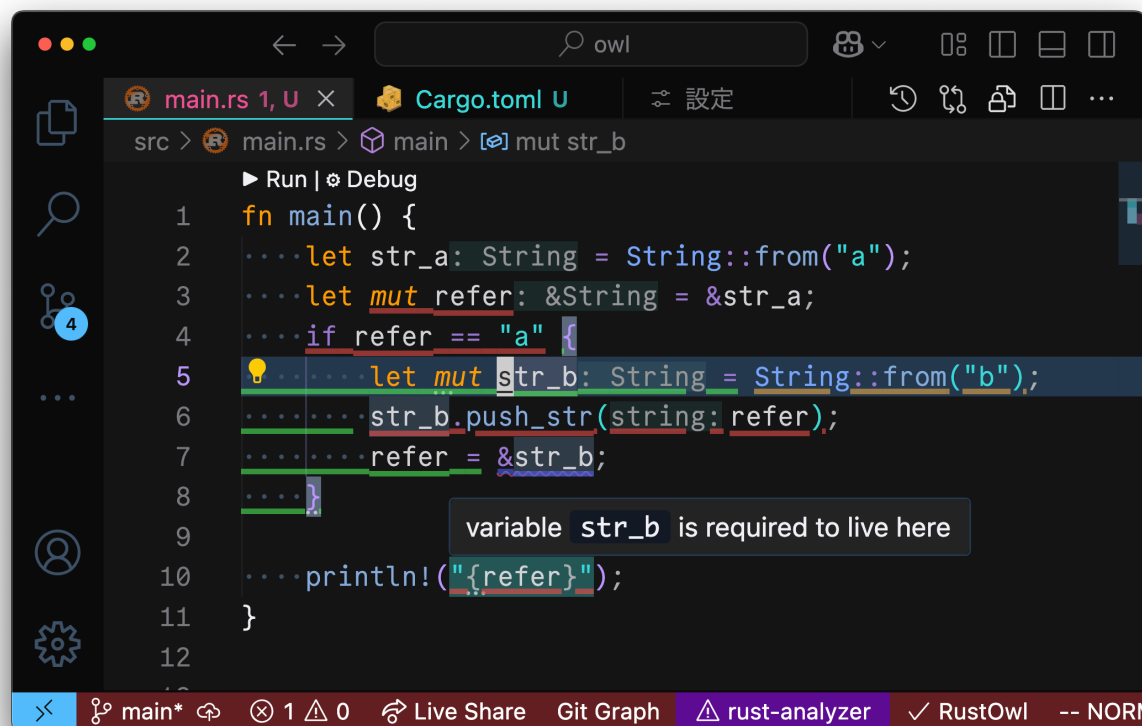
- Target audience: web developers, full-stack developers, systems programmers, cloud developers
- Date: Tuesday December 16, 2025
- Time: 19:00 - 20:00
- Location: DoubleVerify near Flanders Expo
- Registration: <https://www.meetup.com/sysghent/events/312172422>

Events TBA: performance and benchmarking, technical writing with Typst

1.2. New tools

Forked rustowl as ferrous-owl to support Helix and NixOS and add tests (**testers needed**):

- Crates.io: <https://crates.io/crates/ferrous-owl>
- VS Code: <https://marketplace.visualstudio.com/items?itemName=WillemVanhulle.ferrous-owl>



The screenshot shows the Visual Studio Code editor with a Rust project. The file explorer on the left shows 'main.rs' and 'Cargo.toml'. The editor window displays the following Rust code:

```
1 fn main() {  
2     ...let str_a: String = String::from("a");  
3     ...let mut refer: &String = &str_a;  
4     ...if refer == "a" {  
5         ...let mut str_b: String = String::from("b");  
6         ...str_b.push_str(string: refer);  
7         ...refer = &str_b;  
8     }  
9  
10    ...println!("{refer}");  
11 }  
12
```

A tooltip points to line 5, indicating the error: "variable str_b is required to live here". The status bar at the bottom shows "rust-analyzer" and "RustOwl -- NORM".

1. Review tests	1
2. Error handling	4
2.1. Panics	6
2.2. Panics in both debug and release mode	7
2.3. Panics silent in release	8
2.4. Catching panics	9
2.5. Result	10
2.6. Exceptions	11
2.7. Exercises	12
2.8. To unwrap or not to unwrap	13
2.9. Implementing your own Error types	14
2.10. Exercise	15
2.11. Try operator	16
2.12. Exercise	17
2.13. main returning Result	18
2.14. Dynamic Error Types (bad idea)	19
3. Unsafe Rust	20
4. Homework	32

Allocated time: 55 min

In case of a fatal runtime error, Rust triggers a “panic”:

```
1 fn main() {  
2     let v = vec![10, 20, 30];  
3     dbg!(v[100]);  
4 }
```

([playground link](#))

Warning

Panics are for unrecoverable and unexpected errors.

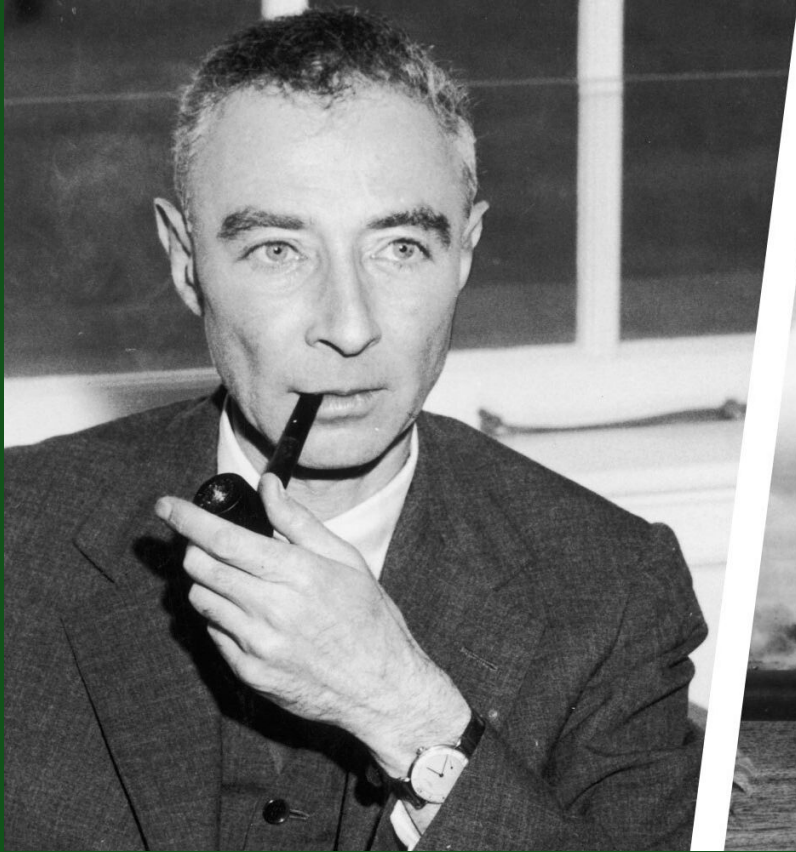
In case of a fatal runtime error, Rust triggers a “panic”:

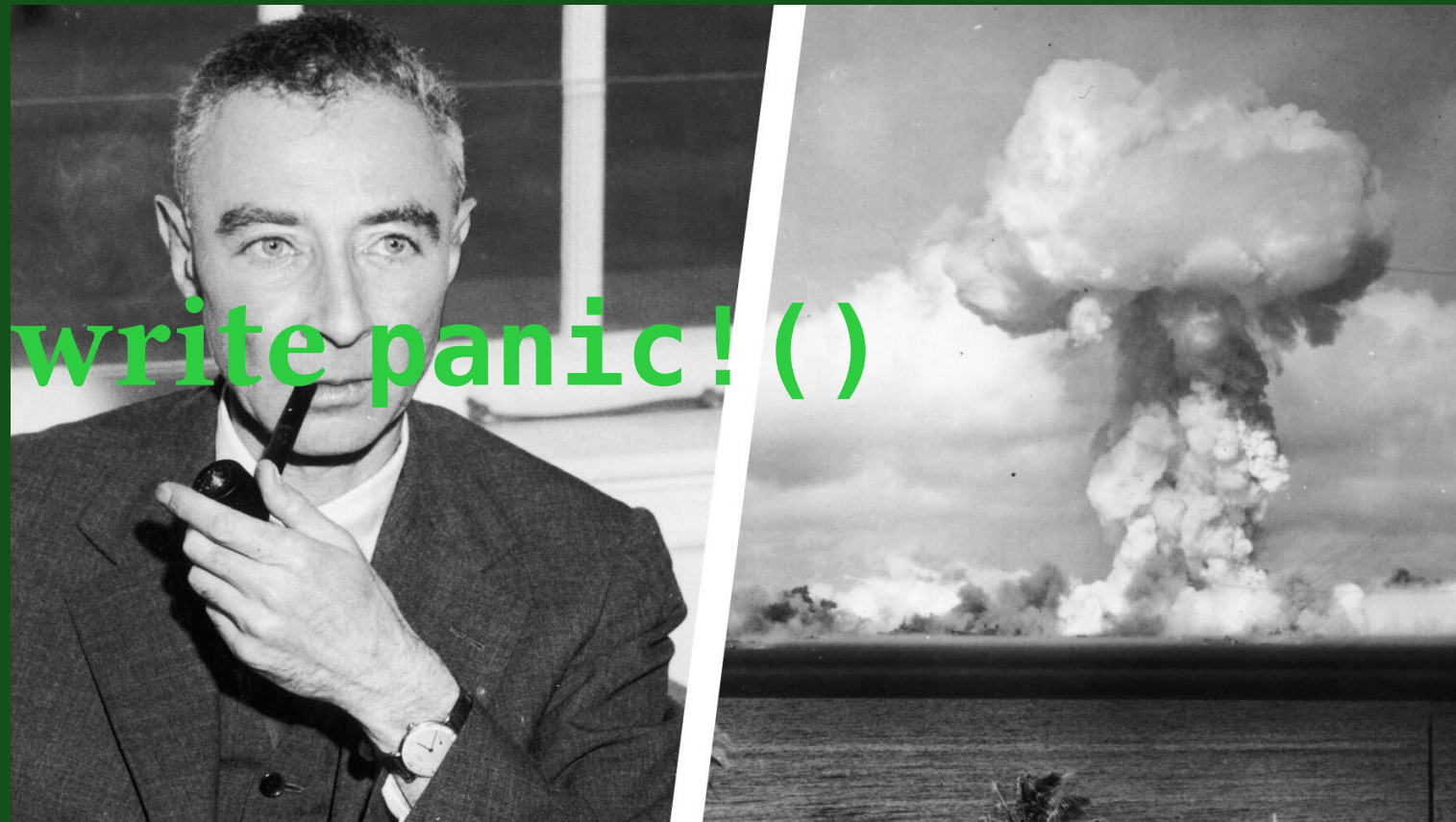
```
1 fn main() {  
2     let v = vec![10, 20, 30];  
3     dbg!(v[100]);  
4 }
```

([playground link](#))

Warning

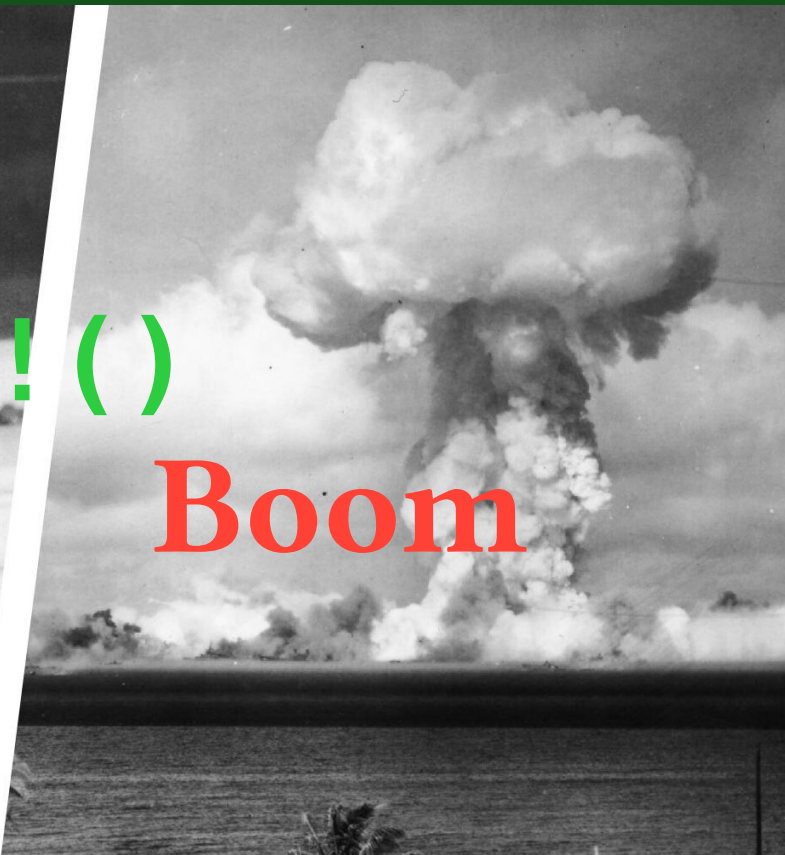
Panics are for unrecoverable and unexpected errors.







write panic!()

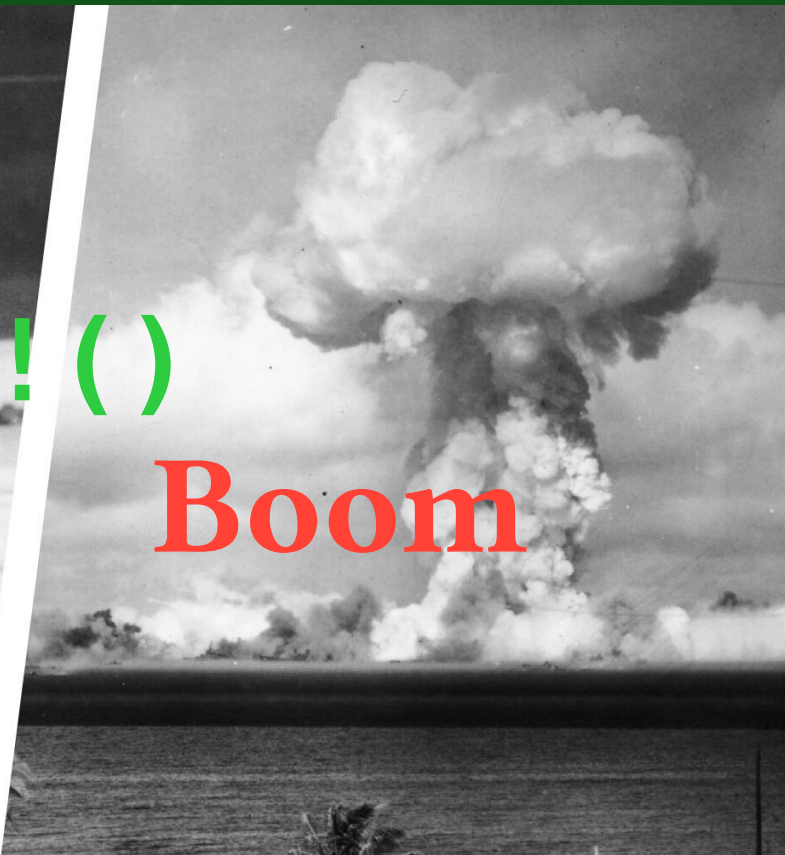


Boom



write panic!()

Panic!



Boom

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

Out-of-bounds array/vector indexing

Name a panic cause related to arithmetic.

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

Out-of-bounds array/vector indexing

Name a panic cause related to arithmetic.

Division by zero

Name 2 panic causes related to Option/Result handling.

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

Out-of-bounds array/vector indexing

Name a panic cause related to arithmetic.

Division by zero

Name 2 panic causes related to Option/Result handling.

1. Calling `unwrap()` on `None`
2. Calling `unwrap()` or `expect()` on `Err`

Name 3 panic causes related to explicit panic macros.

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

Out-of-bounds array/vector indexing

Name a panic cause related to arithmetic.

Division by zero

Name 2 panic causes related to Option/Result handling.

1. Calling `unwrap()` on `None`
2. Calling `unwrap()` or `expect()` on `Err`

Name 3 panic causes related to explicit panic macros.

1. `Explicit panic!()` macro invocation
2. `unreachable!()` macro reached
3. `todo!()` or `unimplemented!()` macro reached

Name a panic cause related to assertions.

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

Out-of-bounds array/vector indexing

Name a panic cause related to arithmetic.

Division by zero

Name 2 panic causes related to Option/Result handling.

1. Calling `unwrap()` on `None`
2. Calling `unwrap()` or `expect()` on `Err`

Name 3 panic causes related to explicit panic macros.

1. `Explicit panic!()` macro invocation
2. `unreachable!()` macro reached
3. `todo!()` or `unimplemented!()` macro reached

Name a panic cause related to assertions.

`assert!()` or `assert_eq!()` failure

Name a panic cause related to runtime limits.

2.2. Panics in both debug and release mode

Name a panic cause related to indexing.

Out-of-bounds array/vector indexing

Name a panic cause related to arithmetic.

Division by zero

Name 2 panic causes related to Option/Result handling.

1. Calling `unwrap()` on `None`
2. Calling `unwrap()` or `expect()` on `Err`

Name 3 panic causes related to explicit panic macros.

1. `Explicit panic!()` macro invocation
2. `unreachable!()` macro reached
3. `todo!()` or `unimplemented!()` macro reached

Name a panic cause related to assertions.

`assert!()` or `assert_eq!()` failure

Name a panic cause related to runtime limits.

Stack overflow from infinite recursion



Bye-bye
debug mode

2.3. Panics silent in release

Why does debug mode have more panics?

2.3. Panics silent in release

Why does debug mode have more panics?

Debug mode prioritizes catching bugs early with runtime checks.

Why does release mode have fewer panics?

2.3. Panics silent in release

Why does debug mode have more panics?

Debug mode prioritizes catching bugs early with runtime checks.

Why does release mode have fewer panics?

Release mode removes some runtime checks to optimize speed.

Name 3 debug-only panics related to arithmetic.

2.3. Panics silent in release

Why does debug mode have more panics?

Debug mode prioritizes catching bugs early with runtime checks.

Why does release mode have fewer panics?

Release mode removes some runtime checks to optimize speed.

Name 3 debug-only panics related to arithmetic.

1. Integer overflow (wraps silently in release)
2. Integer underflow (wraps silently in release)
3. Multiplication overflow (e.g., large `i32 * i32`)

2.3. Panics silent in release

Why does debug mode have more panics?

Debug mode prioritizes catching bugs early with runtime checks.

Why does release mode have fewer panics?

Release mode removes some runtime checks to optimize speed.

Name 3 debug-only panics related to arithmetic.

1. Integer overflow (wraps silently in release)
2. Integer underflow (wraps silently in release)
3. Multiplication overflow (e.g., large `i32 * i32`)

Warning

In release mode, integer overflow wraps around (e.g., `u8::MAX + 1 == 0`). Use `wrapping_*`, `checked_*`, `saturating_*`, or `overflowing_*` methods for explicit control.

Name 3 debug-only panics related to debug assertions.

2.3. Panics silent in release

Why does debug mode have more panics?

Debug mode prioritizes catching bugs early with runtime checks.

Why does release mode have fewer panics?

Release mode removes some runtime checks to optimize speed.

Name 3 debug-only panics related to arithmetic.

1. Integer overflow (wraps silently in release)
2. Integer underflow (wraps silently in release)
3. Multiplication overflow (e.g., large `i32 * i32`)

Warning

In release mode, integer overflow wraps around (e.g., `u8::MAX + 1 == 0`). Use `wrapping_*`, `checked_*`, `saturating_*`, or `overflowing_*` methods for explicit control.

Name 3 debug-only panics related to debug assertions.

1. `debug_assert!()` failure (ignored in release)
2. `debug_assert_eq!()` failure (ignored in release)
3. `debug_assert_ne!()` failure (ignored in release)

2.4. Catching panics

This can be useful in servers which should keep running even if a single request crashes.

```
1  use std::panic;  
2  
3  fn main() {  
4      let result = panic::catch_unwind(|| "No problem here!");  
5      dbg!(result);  
6  
7      let result = panic::catch_unwind(|| {  
8          panic!("oh no!");  
9      });  
10     dbg!(result);  
11 }
```

(playground link)

This does not work if panic = 'abort' is set in your Cargo.toml.

Info

While unwind-ing the stack, destructors are still run for all in-scope variables.

```
1  use std::fs::File; use std::io::Read;
2  fn main() {
3      let file: Result<File, std::io::Error> = File::open("diary.txt");
4      match file {
5          Ok(mut file) => {
6              let mut contents = String::new();
7              if let Ok(bytes) = file.read_to_string(&mut contents) {
8                  println!("Dear diary: {contents} ({bytes} bytes)");
9              } else {
10                 println!("Could not read file content");
11             }
12         }
13         Err(err) => {
14             println!("The diary could not be opened: {err}");
15         }
16     }
17 }
```

(playground link)

2.6. Exceptions

Example of implicit exception in C++:

```
1 void read_file() {  
2     std::ifstream file("diary.txt");  
3     if (!file) throw std::runtime_error("Cannot open file");  
4     // Caller has no indication this function can throw  
5 }  
6  
7 int main() { read_file(); }
```

(playground link)

What are exceptions in other languages the closest to? Abort panics, unwind panics, or results?

2.6. Exceptions

Example of implicit exception in C++:

```
1 void read_file() {  
2     std::ifstream file("diary.txt");  
3     if (!file) throw std::runtime_error("Cannot open file");  
4     // Caller has no indication this function can throw  
5 }  
6  
7 int main() { read_file(); }
```

(playground link)

What are exceptions in other languages the closest to? Abort panics, unwind panics, or results?

Results

What is the difference between a rust Result and exceptions in other languages?

2.6. Exceptions

Example of implicit exception in C++:

```
1 void read_file() {  
2     std::ifstream file("diary.txt");  
3     if (!file) throw std::runtime_error("Cannot open file");  
4     // Caller has no indication this function can throw  
5 }  
6  
7 int main() { read_file(); }
```

(playground link)

What are exceptions in other languages the closest to? Abort panics, unwind panics, or results?

Results

What is the difference between a rust Result and exceptions in other languages?

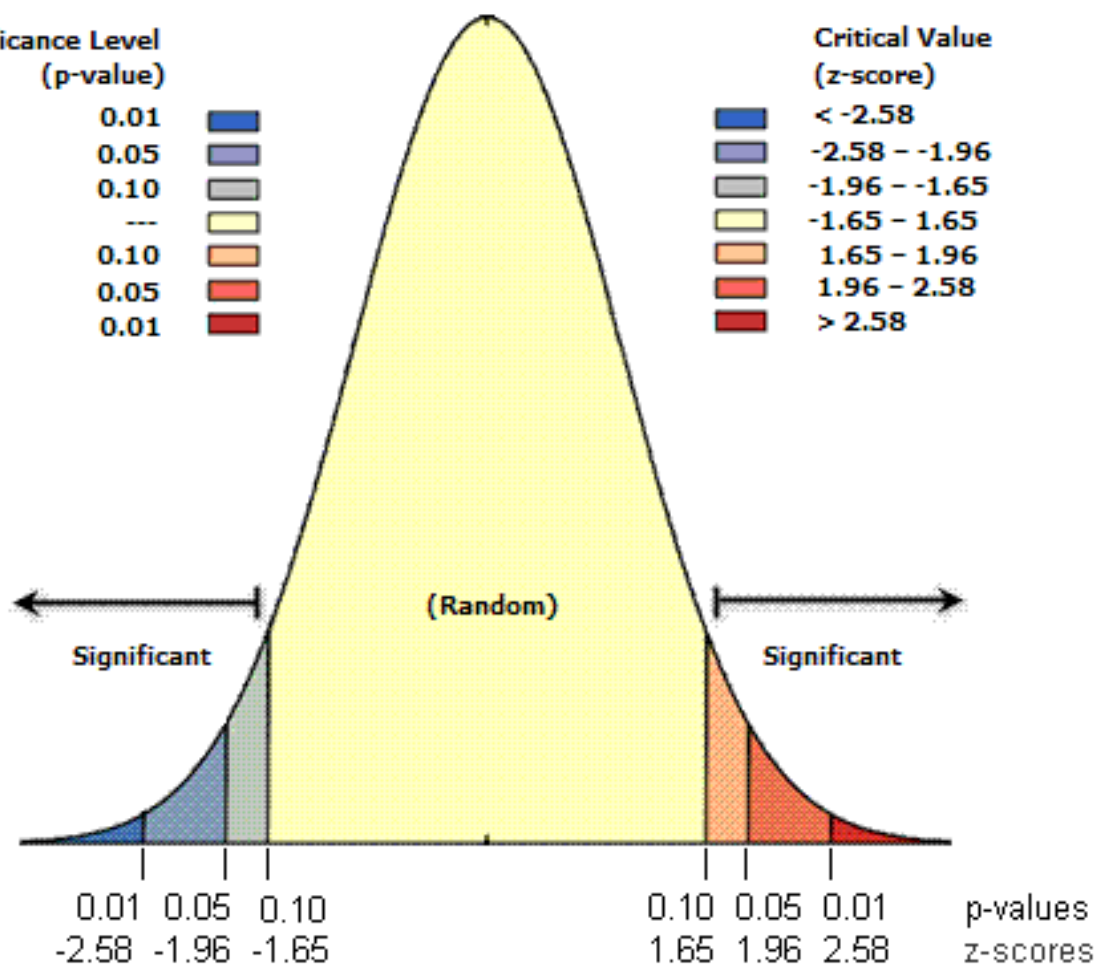
Results must be handled with pattern matching, while exceptions are often implicit.

Significance Level
(p-value)

0.01	Blue
0.05	Light Blue
0.10	Grey
---	Yellow
0.10	Orange
0.05	Red-Orange
0.01	Red

Critical Value
(z-score)

Blue	< -2.58
Light Blue	-2.58 - -1.96
Grey	-1.96 - -1.65
Yellow	-1.65 - 1.65
Orange	1.65 - 1.96
Red-Orange	1.96 - 2.58
Red	> 2.58

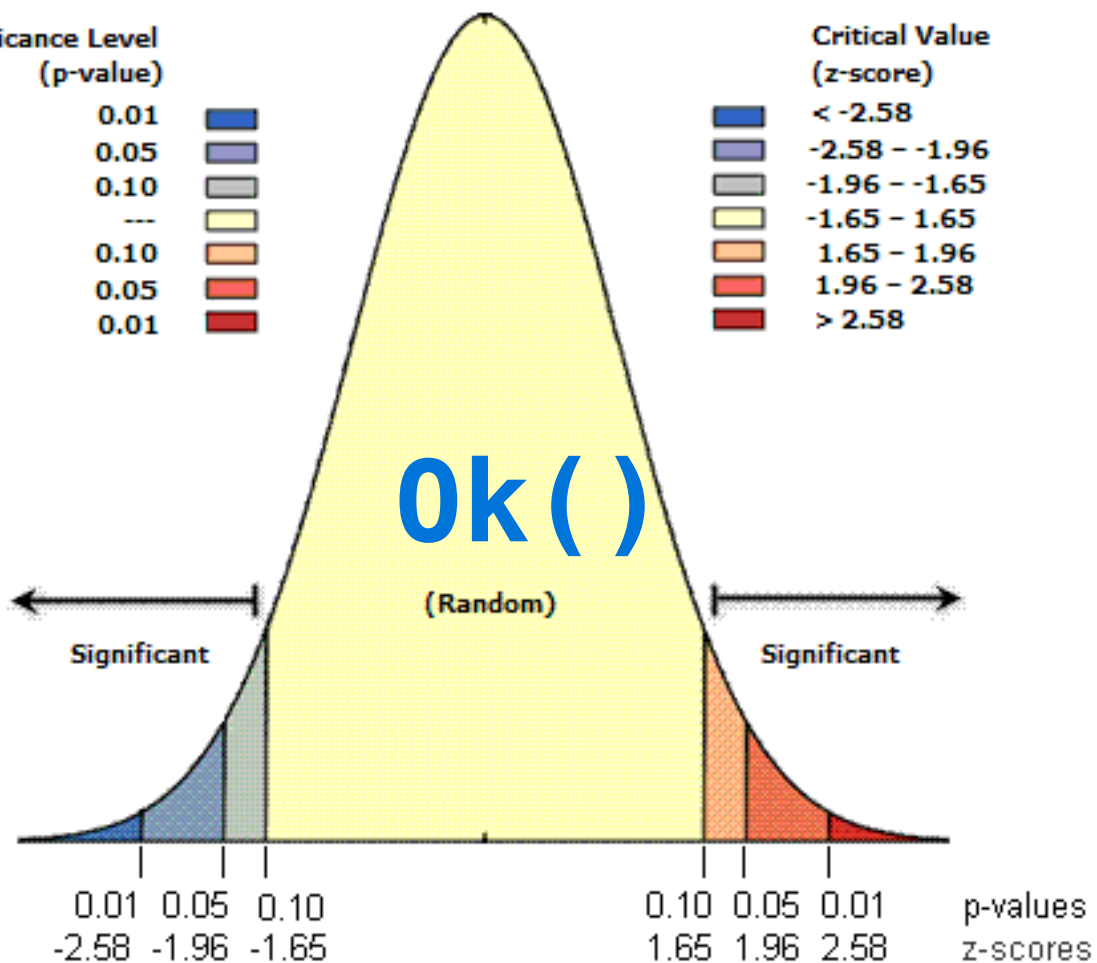


Significance Level
(p-value)

0.01	Blue
0.05	Light Blue
0.10	Grey
---	Yellow
0.10	Orange
0.05	Red-Orange
0.01	Red

Critical Value
(z-score)

Blue	< -2.58
Light Blue	-2.58 - -1.96
Grey	-1.96 - -1.65
Yellow	-1.65 - 1.65
Orange	1.65 - 1.96
Red-Orange	1.96 - 2.58
Red	> 2.58

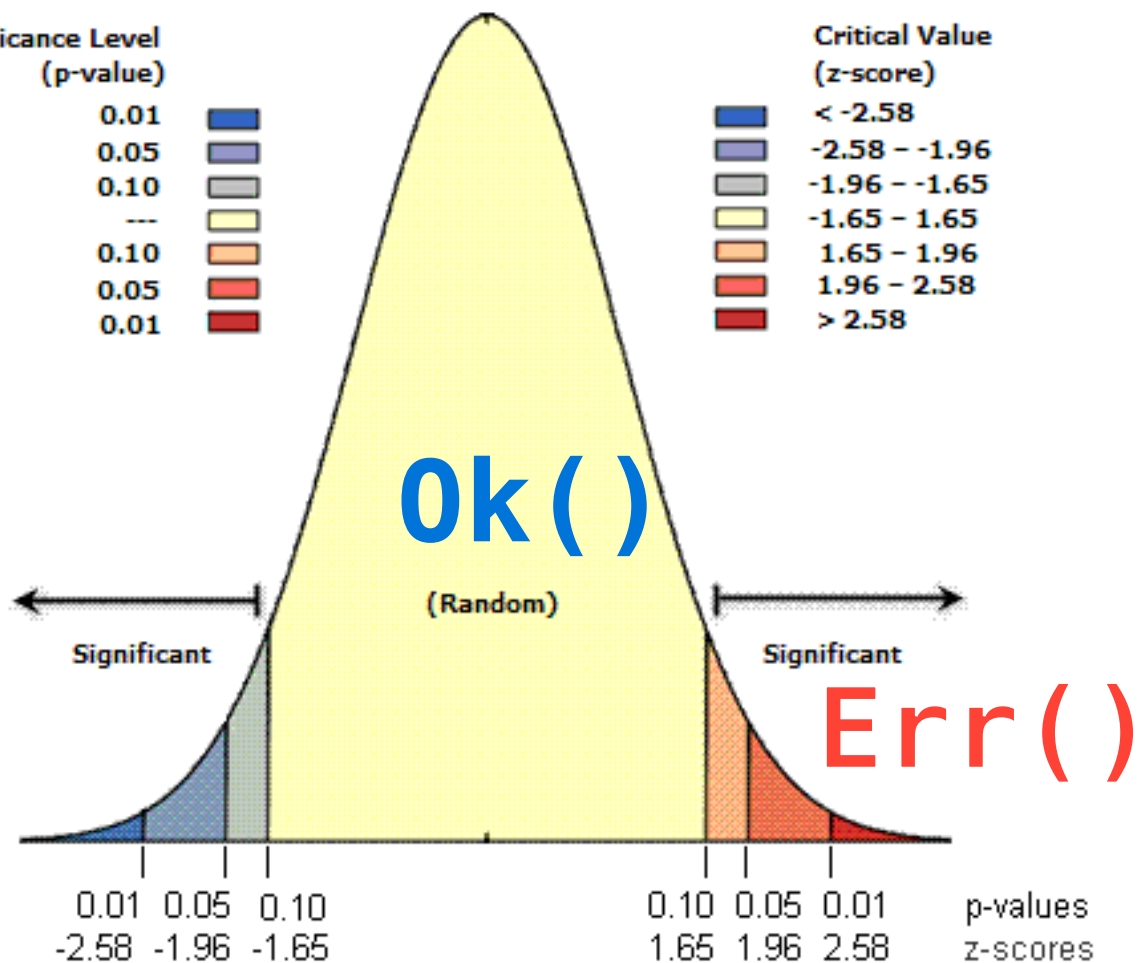


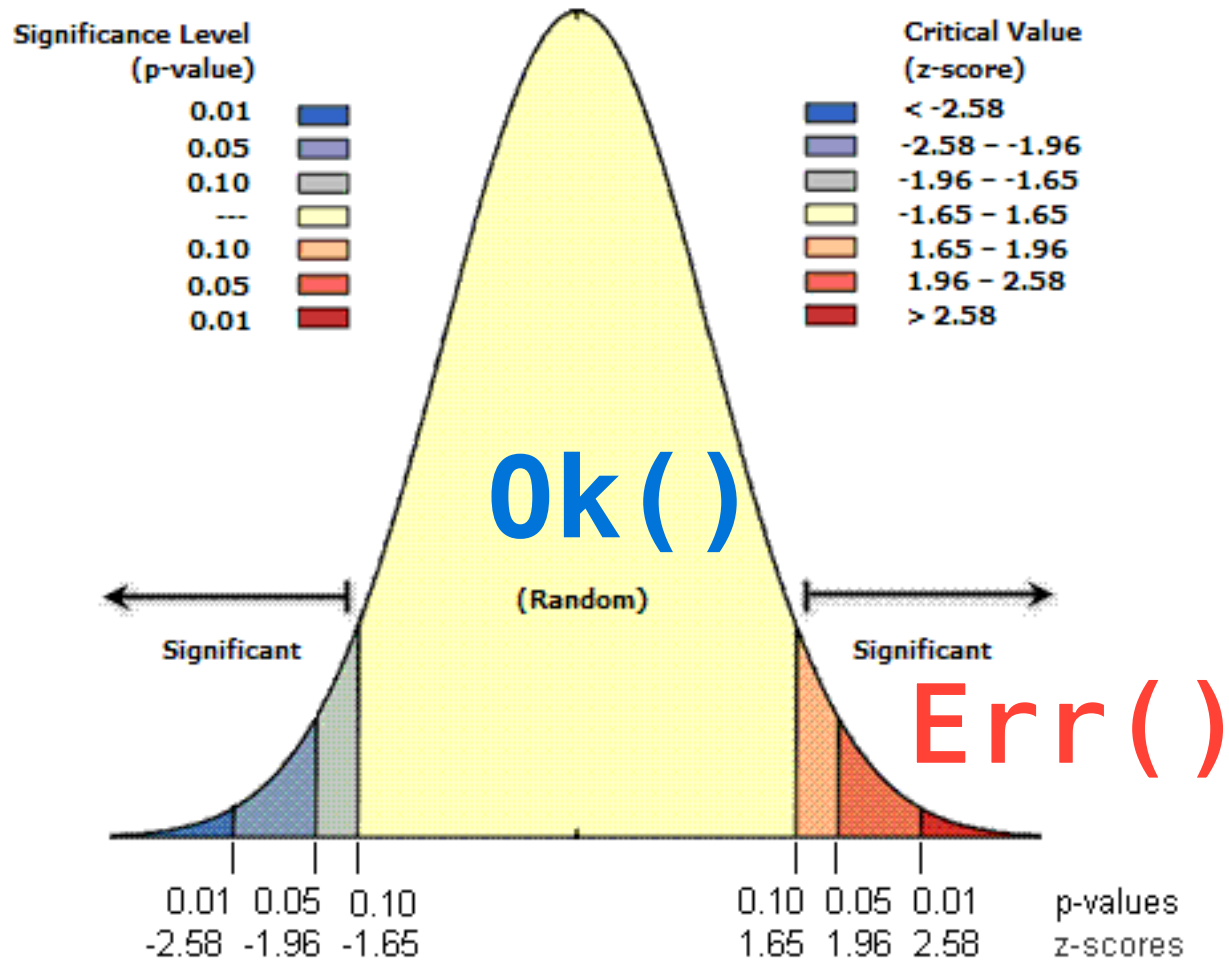
Significance Level
(p-value)

0.01	Blue
0.05	Light Blue
0.10	Grey
---	Yellow
0.10	Orange
0.05	Red-Orange
0.01	Red

Critical Value
(z-score)

Blue	< -2.58
Light Blue	-2.58 - -1.96
Grey	-1.96 - -1.65
Yellow	-1.65 - 1.65
Orange	1.65 - 1.96
Red-Orange	1.96 - 2.58
Red	> 2.58





Err()ors
are a
fact of life

Rustlings, chapter 13 (`error_handling`): ex. 1, 2, 4, 6

2.8. To unwrap or not to unwrap

The `Result` type has a convenience method:

```
1 let file = File::open("diary.txt").unwrap();
```

(playground link)

When is it appropriate to use `unwrap()`?

2.8. To unwrap or not to unwrap

The `Result` type has a convenience method:

```
1 let file = File::open("diary.txt").unwrap();
```

(playground link)

When is it appropriate to use `unwrap()`?

When you are sure the `Result` is `Ok`, such as in tests or examples.

When should never use `unwrap()`?

2.8. To unwrap or not to unwrap

The Result type has a convenience method:

```
1 let file = File::open("diary.txt").unwrap();
```

[\(playground link\)](#)

When is it appropriate to use `unwrap()`?

When you are sure the Result is Ok, such as in tests or examples.

When should never use `unwrap()`?

In production code where the Result could be Err, as it can cause panics.

But what if we add a reason like this?

```
1 let file = File::open("diary.txt").expect("Diary must exist");
```

[\(playground link\)](#)

Why is this a bad idea?

2.8. To unwrap or not to unwrap

The Result type has a convenience method:

```
1 let file = File::open("diary.txt").unwrap();
```

[\(playground link\)](#)

When is it appropriate to use `unwrap()`?

When you are sure the Result is Ok, such as in tests or examples.

When should never use `unwrap()`?

In production code where the Result could be Err, as it can cause panics.

But what if we add a reason like this?

```
1 let file = File::open("diary.txt").expect("Diary must exist");
```

[\(playground link\)](#)

Why is this a bad idea?

You are essentially writing a string comment that refers to an anonymous variable.

2.9. Implementing your own Error types

Instead using `panic!()` or `expect()`, define your own error types.

```
1 #[derive(Debug)]  
2 enum MathError {  
3     DivisionByZero,  
4     NegativeLogarithm,  
5 }  
6  
7 impl Error for MathError {}
```

[\(playground link\)](#)

Whenever you unwrap a `Result::Err<_, MathError>` and get a panic, the Rust binary prints the `MathError` value:

```
1 thread 'main' panicked at 'called `Result::unwrap()` on an `Err` value:  
   DivisionByZero', src/main.rs:10:34
```

[\(playground link\)](#)

Warning

Add error metadata to the error enum variants!

2.10. Exercise

Complete example `examples/custom-error.rs`.

2.11. Try operator

Matching on `Result` on every call can be cumbersome.

2.11. Try operator

Matching on `Result` on every call can be cumbersome.

You can rewrite:

```
1 match some_expression {  
2     Ok(value) => value,  
3     Err(err) => return Err(err),  
4 }
```

([playground link](#))

Into:

2.11. Try operator

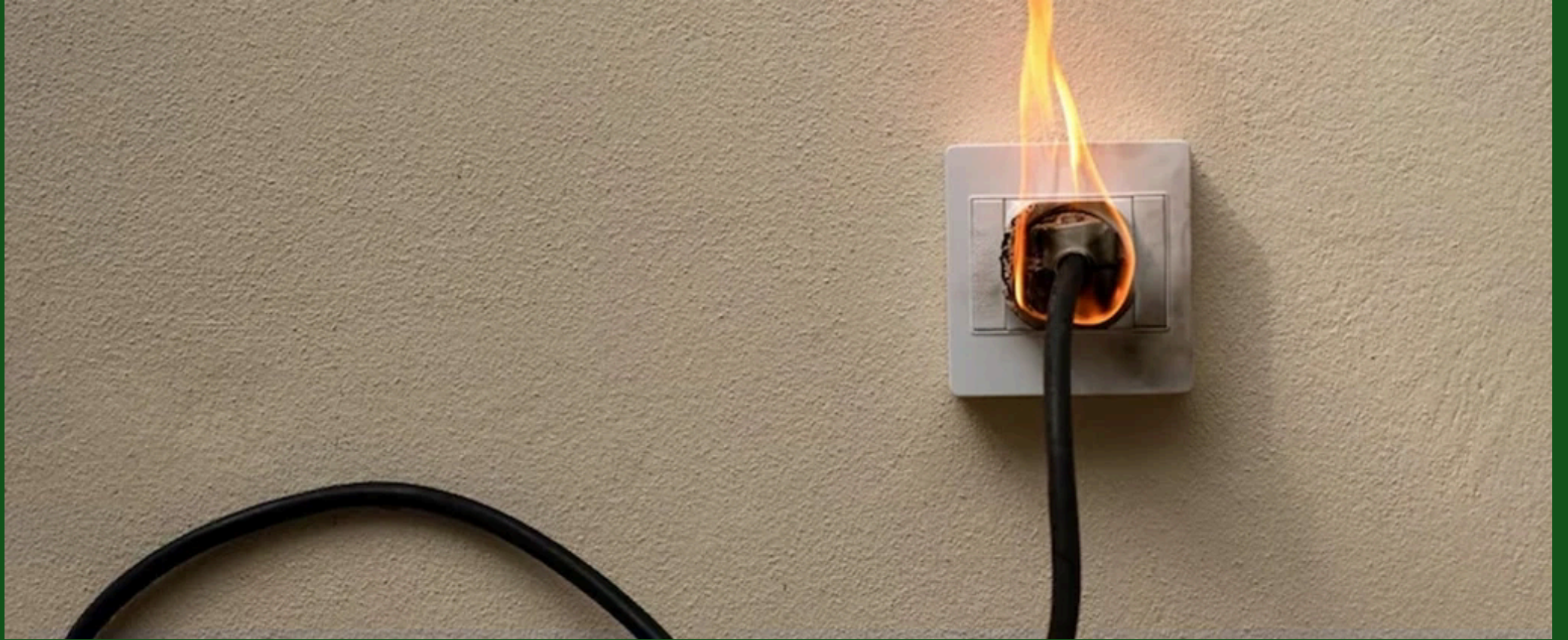
Matching on `Result` on every call can be cumbersome.

You can rewrite:

```
1 match some_expression {  
2     Ok(value) => value,  
3     Err(err) => return Err(err),  
4 }
```

([playground link](#))

Into: `some_expression?.`



2.12. Exercise

Rustlings:

- Ch 13, ex.3, 5

Google:

- Rewrite `examples/short-circuit.rs` to use the `?` operator.
- Fix `tests/error-handling.rs`.

2.13. main returning Result

```
1 fn main() -> Result<(), Error> {}
```

(playground link)

Should you annotate the return type of main to be `Result<(), E>`?

2.13. main returning Result

```
1 fn main() -> Result<(), Error> {}
```

(playground link)

Should you annotate the return type of main to be `Result<(), E>`?

No, because most E implement Debug.

The executable will print the Err variant and return a nonzero exit status on error.

2.13. main returning Result

```
1 fn main() -> Result<(), Error> {}
```

(playground link)

Should you annotate the return type of main to be `Result<(), E>`?

No, because most E implement Debug.

The executable will print the Err variant and return a nonzero exit status on error.

How does this work?

2.13. main returning Result

```
1 fn main() -> Result<(), Error> {}
```

[\(playground link\)](#)

Should you annotate the return type of main to be Result<(), E>?

No, because most E implement Debug.

The executable will print the Err variant and return a nonzero exit status on error.

How does this work?

```
1 impl Termination for Infallible {}
```

```
2 impl Termination for ! {}
```

```
3 impl Termination for () {}
```

```
4 impl Termination for ExitCode {}
```

```
5 impl<T: Termination, E: Debug> Termination for Result<T, E> {}
```

[\(playground link\)](#)

2.14. Dynamic Error Types (bad idea)

```
1  use std::error::Error; use std::fs; use std::io::Read;
2  fn read_count(path: &str) -> Result<i32, Box<dyn Error>> {
3      let mut count_str = String::new();
4      fs::File::open(path)?.read_to_string(&mut count_str)?;
5      let count: i32 = count_str.parse()?;
6      Ok(count)
7  }
8
9  fn main() {
10     fs::write("count.dat", "1i3").unwrap();
11     match read_count("count.dat") {
12         Ok(count) => println!("Count: {count}"),
13         Err(err)  => println!("Error: {err}"),
14     }
15 }
```

(playground link)

[E03] Error: Incompatible types

[sample.tao:1:13]

1 → def five = match () in {

2 () => 5,

·

·

3 () => "5",

·

·

4 → }

·

·

·

·

— The values are outputs of this match expression

Note: Outputs of match expressions must coerce to the same type

1. Review tests	1
2. Error handling	4
3. Unsafe Rust	20
3.1. Unsafe	21
3.2. Safety comments	22
3.3. Incorrect unsafe code	23
3.4. Exercise: Unsafe Pointer Dereferencing	24
3.5. Mutable Static Variables	25
3.6. Unsafe mutation of static variables	26
3.7. Unsafe in no_std environments	27
3.8. Unsafe in embedded	28
3.9. Exercise	30
3.10. Unions	31
4. Homework	32

Allocated time: 30 min

3.1. Unsafe

Unsafe Rust gives you access to five new capabilities:

- Dereference raw pointers.
- Access or modify mutable static variables.
- Access union fields.
- Call unsafe functions, including extern functions.
- Implement unsafe traits.

Unsafe code is usually small and isolated, and its correctness should be carefully documented. It is usually wrapped in a safe abstraction layer.

What is undefined behaviour (UB) in Rust?

3.1. Unsafe

Unsafe Rust gives you access to five new capabilities:

- Dereference raw pointers.
- Access or modify mutable static variables.
- Access union fields.
- Call unsafe functions, including extern functions.
- Implement unsafe traits.

Unsafe code is usually small and isolated, and its correctness should be carefully documented. It is usually wrapped in a safe abstraction layer.

What is undefined behaviour (UB) in Rust?

When the compiler assumes certain invariants hold, but they are violated at runtime, leading to unpredictable behavior.

What is the relationship between unsafe code and undefined behaviour?

3.1. Unsafe

Unsafe Rust gives you access to five new capabilities:

- Dereference raw pointers.
- Access or modify mutable static variables.
- Access union fields.
- Call unsafe functions, including extern functions.
- Implement unsafe traits.

Unsafe code is usually small and isolated, and its correctness should be carefully documented. It is usually wrapped in a safe abstraction layer.

What is undefined behaviour (UB) in Rust?

When the compiler assumes certain invariants hold, but they are violated at runtime, leading to unpredictable behavior.

What is the relationship between unsafe code and undefined behaviour?

Unsafe code likely leads to undefined behavior.



3.2. Safety comments

```
1  fn main() {
2      let mut x = 10;
3
4      let p1: *mut i32 = &raw mut x;
5      let p2 = p1 as *const i32;
6
7      // SAFETY: p1 and p2 were created by taking raw pointers to a local, so they
8      // are guaranteed to be non-null, aligned, and point into a single (stack-)
9      // allocated object.
10     unsafe {
11         dbg!(*p1);
12         *p1 = 6;
13         // Mutation may soundly be observed through a raw pointer, like in C.
14         dbg!(*p2);
15     }
16
17 }
```

(playground link)

```
1  fn main() {
2      let mut x = 10;
3
4      let p1: *mut i32 = &raw mut x;
5      let p2 = p1 as *const i32;
6
7      // UNSOUND. DO NOT DO THIS.
8      /*
9      let r: &i32 = unsafe { &*p1 };
10     dbg!(r);
11     x = 50;
12     dbg!(r); // Object underlying the reference has been mutated. This is UB.
13     */
14 }
```

(playground link)

3.4. Exercise: Unsafe Pointer Dereferencing

Complete `examples/unsafe-pointers.rs`

Run tests: `cargo test --example unsafe-pointers`

3.5. Mutable Static Variables

It is safe to read an immutable static variable:

```
1 static HELLO_WORLD: &str = "Hello, world!";  
2  
3 fn main() {  
4     println!("HELLO_WORLD: {HELLO_WORLD}");  
5 }
```

(playground link)

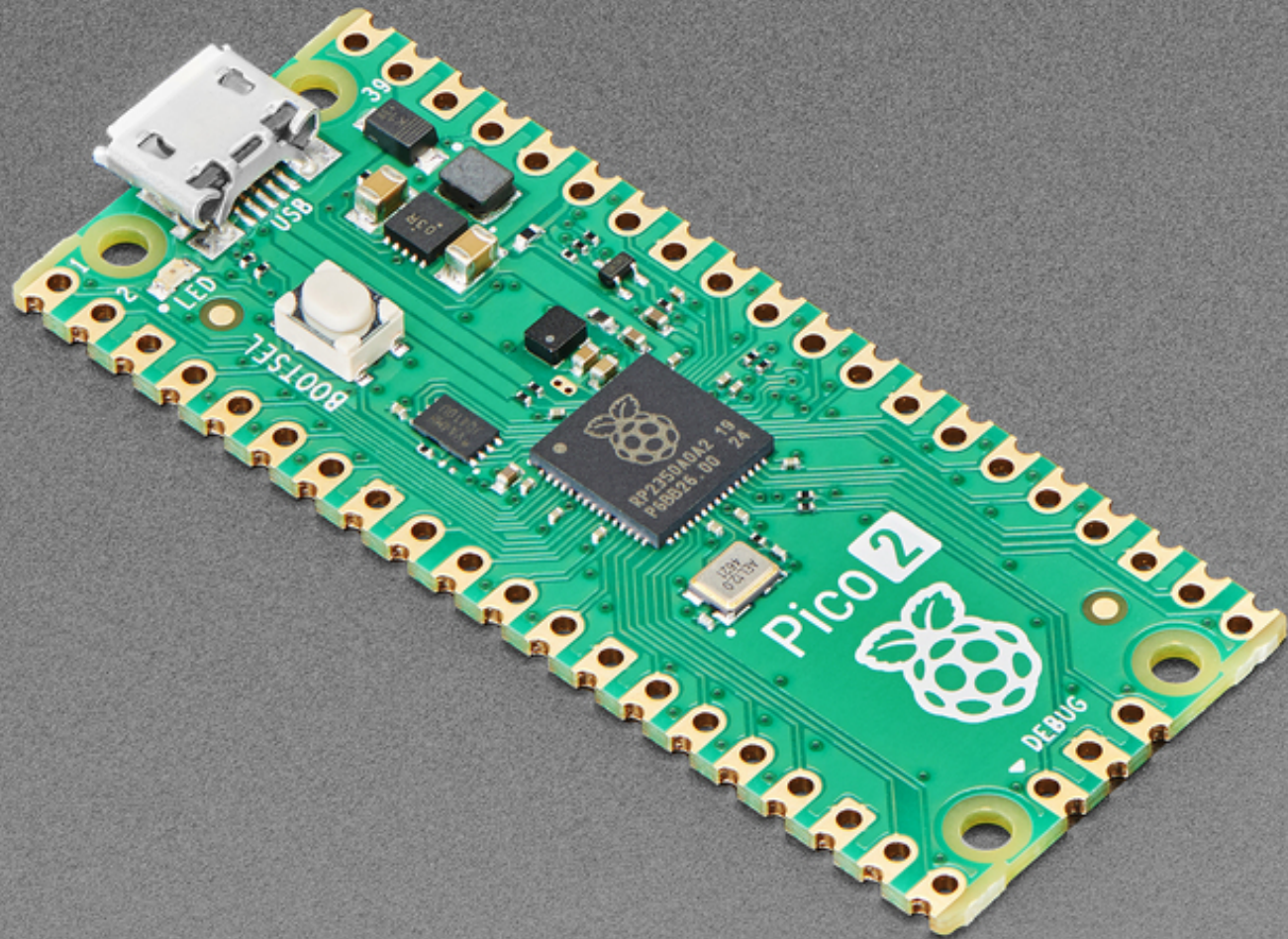
3.6. Unsafe mutation of static variables

Using mutable statics soundly requires reasoning about concurrency without the compiler's help:

```
1  static mut COUNTER: u32 = 0;
2  fn add_to_counter(inc: u32) {
3      // SAFETY: There are no other threads which could be accessing `COUNTER`.
4      unsafe {
5          COUNTER += inc;
6      }
7  }
8  fn main() {
9      add_to_counter(42);
10     // SAFETY: There are no other threads which could be accessing `COUNTER`.
11     unsafe {
12         dbg!(COUNTER);
13     }
14 }
```

(playground link)

May cause race-conditions if multiple threads access the static simultaneously.



What does no_std mean?

3.7. Unsafe in `no_std` environments

What does `no_std` mean?

A Rust environment without the standard library and optional heap allocator.

Remember: `std` > `alloc` > `core`

```
1  #![no_std]
2  // Raw pointer to memory-mapped I/O register
3  static GPIO_OUTPUT: *mut u32 = 0x4000_5000 as *mut u32;
4
5  fn set_pin_high(pin: u8) {
6      // SAFETY: GPIO_OUTPUT points to a valid memory-mapped register
7      // and we have exclusive access during startup.
8      unsafe {
9          GPIO_OUTPUT.write_volatile(1 << pin);
10     }
11 }
```

(playground link)

Info

A lot of unsafe in embedded code is abstracted away in Hardware Abstraction Layer (HAL) crates.

Do raw pointers replace references in `no_std` code?

3.8. Unsafe in embedded

Do raw pointers replace references in `no_std` code?

No. Higher-level `no_std` code still uses references normally. Raw pointers are only necessary when interfacing with hardware or memory regions where the compiler cannot verify validity.

Which validity guarantees do references require that can't be verified for hardware addresses?

3.8. Unsafe in embedded

Do raw pointers replace references in `no_std` code?

No. Higher-level `no_std` code still uses references normally. Raw pointers are only necessary when interfacing with hardware or memory regions where the compiler cannot verify validity.

Which validity guarantees do references require that can't be verified for hardware addresses?

References must be non-null, properly aligned, and point to initialized memory. For hardware addresses, the compiler cannot verify these properties, so raw pointers are used instead.

3.9. Exercise

Try to flash a simple blink program using the HAL crate one of the supported boards.

Warning

Only use the processor or HAL crate for the exercises, no high-level frameworks like Embassy!

Ask for boards and help if needed.

See also my SysGhent workshop for building a smart plant pot with embedded Rust: <https://github.com/sysghent/plant-pot>

3.9.1. Raspberry Pi Pico

For example, you could flash a Raspberry Pi Pico with <https://github.com/rp-rs/rp-hal>.

You can find example programs on <https://github.com/rp-rs/rp-hal/tree/main/rp2040-hal-examples/src/bin>

3.9.2. ESP32

Have a look at <https://github.com/esp-rs/esp-hal>

3.10. Unions

Unions are like enums, but you need to track the active field yourself:

```
1  #[repr(C)]
2  union MyUnion {
3      i: u8,
4      b: bool,
5  }
6
7  fn main() {
8      let u = MyUnion { i: 42 };
9      println!("int: {}", unsafe { u.i });
10     println!("bool: {}", unsafe { u.b }); // Undefined behavior!
11 }
```

(playground link)

3.10. Unions

Unions are like enums, but you need to track the active field yourself:

```
1  #[repr(C)]
2  union MyUnion {
3      i: u8,
4      b: bool,
5  }
6
7  fn main() {
8      let u = MyUnion { i: 42 };
9      println!("int: {}", unsafe { u.i });
10     println!("bool: {}", unsafe { u.b }); // Undefined behavior!
11 }
```

(playground link)

Occasionally needed for interacting with C library APIs.

3.10. Unions

Unions are like enums, but you need to track the active field yourself:

```
1  #[repr(C)]
2  union MyUnion {
3      i: u8,
4      b: bool,
5  }
6
7  fn main() {
8      let u = MyUnion { i: 42 };
9      println!("int: {}", unsafe { u.i });
10     println!("bool: {}", unsafe { u.b }); // Undefined behavior!
11 }
```

(playground link)

Occasionally needed for interacting with C library APIs.

If you just want to reinterpret bytes as a different type, you probably want `std::mem::transmute`.

1. Review tests	1
2. Error handling	4
3. Unsafe Rust	20
4. Homework	32
4.1. Events coming up	33
4.2. Homework	34

4.1. Events coming up

Scripting with Nu Shell:

- Target audience: DevOps, sysadmins, backend developers
- Date: **tomorrow**, Wednesday December 3, 2025
- Time: 19:00 - 20:00
- Location: Kammerstraat 10, 9000 Gent
- Registration: <https://www.meetup.com/sysghent/events/311799711>

```
1 ps | where name == Notepad2.exe | get pid.0 | kill $in
```

(playground link)

4.1. Events coming up

Scripting with Nu Shell:

- Target audience: DevOps, sysadmins, backend developers
- Date: **tomorrow**, Wednesday December 3, 2025
- Time: 19:00 - 20:00
- Location: Kammerstraat 10, 9000 Gent
- Registration: <https://www.meetup.com/sysghent/events/311799711>

```
1 ps | where name == Notepad2.exe | get pid.0 | kill $in
```

(playground link)

WebAssembly for secure systems and services:

- Target audience: web developers, full-stack developers, systems programmers, cloud developers
- Date: Tuesday December 16, 2025
- Time: 19:00 - 20:00
- Location: DoubleVerify near Flanders Expo
- Registration: <https://www.meetup.com/sysghent/events/312172422>

Events TBA: performance and benchmarking, technical writing with Typst

4.2. Homework

4.2.1. Projects

Final reminder for student projects:

- Only Rust code, no markdown files
- **No AI**
- **Deadline: this Thursday, Dec. 4**

Please open a PR and add me as reviewer. My e-mail is willemvanhulle@protonmail.com and GitHub ID is “wvhulle”

4.2. Homework

4.2.1. Projects

Final reminder for student projects:

- Only Rust code, no markdown files
- **No AI**
- **Deadline: this Thursday, Dec. 4**

Please open a PR and add me as reviewer. My e-mail is willemvanhulle@protonmail.com and GitHub ID is “wvhulle”

4.2.2. Reading

Review:

- Error handling: <https://doc.rust-lang.org/book/ch09-00-error-handling.html>
- Unsafe Rust:
 - Very good read: last chapter in “Programming Rust” by Blandy
 - Unfinished chapters in <https://google.github.io/comprehensive-rust/bare-metal/>

Prepare for next session about parallel programming, **it will be mind-blowing!**